CBSE ARTIFICIAL INTELLIGENCE (417)

CLASS X PROJECT REPORT

# STUDYWISE AI

SMART STUDY PLANNER & REVISION PRIORITY ASSISTANT

**PLAN • PRIORITISE • PROGRESS**

A Python-powered rule-based AI application


**SUBMITTED BY**

## Shivam Srivastava

Class: X-A | Roll No.: 28

School: Sarala Birla Public School


**ACADEMIC SESSION 2026-2027**

# 1. INTRODUCTION

**Project overview.** StudyWise AI is a small desktop application that helps a student organise revision topics before an examination. The student enters a subject, topic, days remaining, difficulty, confidence level and exam importance. The application analyses these inputs and labels each topic as HIGH, MEDIUM or LOW priority.

**Problem statement.** Students often have several chapters to revise but may not know which one needs attention first. Choosing topics only by habit can lead to difficult or important chapters being postponed. StudyWise AI provides a simple, explainable way to organise revision.

**Objectives.**

- Create an easy-to-use desktop interface with Python as the core programming language.
- Collect useful study-planning inputs and check that the values are valid.
- Apply a transparent rule-based AI scoring method to rank topics.
- Generate a daily plan that stays within the student's available study time.
- Save the topic list locally so it is available the next time the app opens.

**AI project cycle.**

- **1. Problem scoping:** The target users are school students who need help organising revision. The problem is deciding which topic to revise first.
- **2. Data acquisition:** The student enters days remaining, difficulty, confidence and exam importance for each topic.
- **3. Data exploration:** The program checks the input ranges and converts the days remaining into an urgency level.
- **4. Modelling:** A rule-based expert system combines four factors into a score and classifies each topic.
- **5. Evaluation:** The scoring rules are checked with sample High, Medium and Low cases. Invalid values are also tested.

**Working principle.** The score is calculated as: 0.30 x Urgency + 0.25 x Difficulty + 0.25 x Knowledge Gap + 0.20 x Exam Importance. Knowledge Gap equals 6 minus Confidence. A score of 4.00 or more is HIGH, 3.00 to 3.99 is MEDIUM, and below 3.00 is LOW.

**Technologies used.** Python 3 is the core programming language for the application logic and data handling. Tkinter provides the desktop graphical interface, while the built-in json, datetime and pathlib modules support local storage and date handling. The current version is a desktop app; HTML, CSS and JavaScript could be used for a future browser-based interface, but they are optional and are not a restriction on the project.

# 2. ACKNOWLEDGEMENT

I express my sincere gratitude to Sarala Birla Public School for giving me the opportunity to complete this project. This work helped me explore how logical rules and Python programming can be combined to solve a practical problem.

I would also like to thank my family and everyone who supported me while planning, developing and testing StudyWise AI. This project helped me practise problem scoping, input validation, algorithm design, user-interface development and responsible use of technology.

Finally, I acknowledge the official Python documentation and CBSE learning resources that helped me understand the tools and concepts used in this project.

**PROJECT**

## STUDYWISE AI

Smart Study Planner & Revision Priority Assistant

# 3. CERTIFICATE

This is to certify that **Shivam Srivastava**, a student of **Class X-A** at **Sarala Birla Public School**, has prepared the Artificial Intelligence project titled **"StudyWise AI - Smart Study Planner & Revision Priority Assistant"** for the academic session **2026-2027**, as part of the CBSE Artificial Intelligence (417) project work.

**Student UID:** SBPS/16-17/3828

The project demonstrates a Python-based desktop application that uses a rule-based decision system to help students organise revision topics. This report presents the project overview, implementation, sample outputs and conclusions.

Subject Teacher's Signature

External Examiner's Signature

Date

School Seal

# 4. SOURCE CODE

StudyWise_AI_Project.py

```
"""StudyWise AI - a rule-based smart study planner.

Run:
    python3 StudyWise_AI_Project.py

Python is the core language for the application logic, interface and data handling.
The current desktop interface uses Tkinter, and study topics are saved locally
in studywise_data.json in the same folder as this script. The design can be
extended with a web interface using HTML/CSS/JavaScript if needed.
"""

import json
import sys
from datetime import date
from pathlib import Path
import tkinter as tk
from tkinter import ttk, messagebox

APP_DIR = Path(__file__).resolve().parent
DATA_FILE = APP_DIR / "studywise_data.json"


# -------------------------- AI DECISION ENGINE --------------------------
def analyse_topic(days_left, difficulty, confidence, importance):
    """Return a priority score, category, explanation and study advice.

    All input values are on a 1-5 scale, except days_left.
    The low-confidence score is higher when the student's confidence is lower.
    """
    if days_left < 0:
        raise ValueError("Days remaining cannot be negative.")
    for value, label in (
        (difficulty, "Difficulty"),
        (confidence, "Confidence"),
        (importance, "Exam importance"),
    ):
        if not 1 <= value <= 5:
            raise ValueError(f"{label} must be between 1 and 5.")

    # Rule 1: convert the time remaining into an urgency score.
    if days_left <= 1:
        urgency = 5
    elif days_left <= 3:
        urgency = 4
    elif days_left <= 7:
        urgency = 3
    elif days_left <= 14:
        urgency = 2
    else:
        urgency = 1

    # Rule 2: combine four factors using transparent weights.
    knowledge_gap = 6 - confidence
    score = round(
        0.30 * urgency
        + 0.25 * difficulty
        + 0.25 * knowledge_gap
        + 0.20 * importance,
        2,
    )

    # Rule 3: classify the topic using the final score.
    if score >= 4.0:
        priority = "HIGH"
        advice = "Revise today for 40 minutes, then solve 5 practice questions."
    elif score >= 3.0:
        priority = "MEDIUM"
        advice = "Schedule a 25-minute revision session within the next 2 days."
    else:
        priority = "LOW"
        advice = "Review this topic for 15 minutes during weekly revision."

    reasons = []
    if urgency >= 4:
        reasons.append("the exam is close")
    if difficulty >= 4:
        reasons.append("the topic is difficult")
    if confidence <= 2:
        reasons.append("confidence is low")
    if importance >= 4:
        reasons.append("exam importance is high")
    explanation = ", ".join(reasons) if reasons else "the topic has a manageable revision profile"

    return {
        "score": score,
        "priority": priority,
        "advice": advice,
        "explanation": explanation,
    }


```

# 4. SOURCE CODE - CONTINUED

StudyWise_AI_Project.py

```
# ------------------------------ MAIN APP ---------------------------------
class StudyWiseApp:
    def __init__(self, root):
        self.root = root
        self.root.title("StudyWise AI | Smart Study Planner")
        self.root.geometry("1180x760")
        self.root.minsize(980, 680)
        self.root.configure(bg="#0b1220")
        self.topics = self.load_topics()
        self.build_styles()
        self.build_interface()
        self.refresh_topics()

        if "--demo" in sys.argv:
            self.load_demo_data()
            self.refresh_topics()
            if self.topics:
                first = max(self.topics, key=lambda item: item["score"])
                self.tree.selection_set(str(first["id"]))
                self.show_recommendation(first)
        if "--demo-plan" in sys.argv:
            self.root.after(900, self.generate_plan)

    def build_styles(self):
        style = ttk.Style()
        style.theme_use("clam")
        style.configure("TFrame", background="#0b1220")
        style.configure("Panel.TFrame", background="#111c2f")
        style.configure("TLabel", background="#0b1220", foreground="#e5edf8", font=("Arial", 10))
        style.configure("Title.TLabel", background="#0b1220", foreground="#f8fafc", font=("Arial", 22, "bold"))
        style.configure("Muted.TLabel", background="#0b1220", foreground="#94a3b8", font=("Arial", 9))
        style.configure("PanelTitle.TLabel", background="#111c2f", foreground="#f8fafc", font=("Arial", 11, "bold"))
        style.configure("CardValue.TLabel", background="#111c2f", foreground="#f8fafc", font=("Arial", 20, "bold"))
        style.configure("CardCaption.TLabel", background="#111c2f", foreground="#94a3b8", font=("Arial", 9))
        style.configure("TEntry", fieldbackground="#1e293b", foreground="#f8fafc", insertcolor="#f8fafc", padding=7)
        style.configure("TCombobox", fieldbackground="#1e293b", background="#1e293b", foreground="#f8fafc", padding=6)
        style.map("TCombobox", fieldbackground=[("readonly", "#1e293b")], foreground=[("readonly", "#f8fafc")])
        style.configure("Accent.TButton", background="#38bdf8", foreground="#082f49", font=("Arial", 10, "bold"), padding=9)
        style.map("Accent.TButton", background=[("active", "#7dd3fc")])
        style.configure("Secondary.TButton", background="#26364d", foreground="#e2e8f0", padding=8)
        style.map("Secondary.TButton", background=[("active", "#334b68")])
        style.configure("Treeview", background="#142238", fieldbackground="#142238", foreground="#e2e8f0", rowheight=30, borderwidth=0,
        font=("Arial", 9))
        style.configure("Treeview.Heading", background="#1e293b", foreground="#cbd5e1", font=("Arial", 9, "bold"), padding=8)
        style.map("Treeview", background=[("selected", "#075985")], foreground=[("selected", "#ffffff")])

    def build_interface(self):
        outer = ttk.Frame(self.root, padding=(22, 18))
        outer.pack(fill="both", expand=True)

        header = ttk.Frame(outer)
        header.pack(fill="x", pady=(0, 16))
        ttk.Label(header, text="StudyWise AI", style="Title.TLabel").pack(anchor="w")
        ttk.Label(
            header,
            text="SMART STUDY PLANNER  |  Rule-based revision priority assistant",
            style="Muted.TLabel",
        ).pack(anchor="w", pady=(3, 0))
        ttk.Label(header, text=date.today().strftime("%d %B %Y"), style="Muted.TLabel").place(relx=1.0, rely=0.25, anchor="ne")

        stats = ttk.Frame(outer)
        stats.pack(fill="x", pady=(0, 16))
        self.stat_total = self.make_stat_card(stats, "TOTAL TOPICS", "0")
        self.stat_high = self.make_stat_card(stats, "HIGH PRIORITY", "0")
        self.stat_done = self.make_stat_card(stats, "COMPLETED", "0")
        self.stat_pending = self.make_stat_card(stats, "STILL TO REVISE", "0")

        content = ttk.Frame(outer)
        content.pack(fill="both", expand=True)
        content.columnconfigure(0, weight=0, minsize=300)
        content.columnconfigure(1, weight=1)
        content.rowconfigure(0, weight=1)

        self.build_form(content)
        self.build_dashboard(content)

    def make_stat_card(self, parent, caption, value):
        card = ttk.Frame(parent, style="Panel.TFrame", padding=(14, 11))
        card.pack(side="left", fill="x", expand=True, padx=(0, 9))
        value_label = ttk.Label(card, text=value, style="CardValue.TLabel")
        value_label.pack(anchor="w")
        ttk.Label(card, text=caption, style="CardCaption.TLabel").pack(anchor="w", pady=(2, 0))
        return value_label

    def build_form(self, parent):
        panel = ttk.Frame(parent, style="Panel.TFrame", padding=16)
        panel.grid(row=0, column=0, sticky="nsew", padx=(0, 14))
        ttk.Label(panel, text="ADD A STUDY TOPIC", style="PanelTitle.TLabel").pack(anchor="w", pady=(0, 12))

        self.subject_var = tk.StringVar()
        self.topic_var = tk.StringVar()
```

# 4. SOURCE CODE - CONTINUED

StudyWise_AI_Project.py

```
        self.days_var = tk.StringVar(value="7")
        self.difficulty_var = tk.StringVar(value="3")
        self.confidence_var = tk.StringVar(value="3")
        self.importance_var = tk.StringVar(value="3")
        self.minutes_var = tk.StringVar(value="90")

        self.add_field(panel, "Subject", self.subject_var, "e.g. Mathematics")
        self.add_field(panel, "Topic / Chapter", self.topic_var, "e.g. Quadratic Equations")
        self.add_field(panel, "Days left until exam", self.days_var, "Number of days")
        self.add_choice(panel, "Difficulty (1 = easy, 5 = hard)", self.difficulty_var)
        self.add_choice(panel, "Confidence (1 = low, 5 = high)", self.confidence_var)
        self.add_choice(panel, "Exam importance (1-5)", self.importance_var)
        self.add_field(panel, "Available study time per day (minutes)", self.minutes_var, "e.g. 90")

        ttk.Button(panel, text="+  ADD TOPIC & ANALYSE", style="Accent.TButton", command=self.add_topic).pack(fill="x", pady=(14, 8))
        ttk.Button(panel, text="LOAD SAMPLE TOPICS", style="Secondary.TButton", command=self.load_demo_action).pack(fill="x")
        ttk.Label(
            panel,
            text="Your entries are stored locally on this computer. No account or internet connection is needed.",
            style="Muted.TLabel",
            wraplength=255,
            justify="left",
        ).pack(anchor="w", pady=(18, 0))

    def add_field(self, parent, label, variable, hint):
        ttk.Label(parent, text=label, style="PanelTitle.TLabel").pack(anchor="w", pady=(7, 4))
        entry = ttk.Entry(parent, textvariable=variable)
        entry.pack(fill="x")
        entry.insert(0, "") if False else None
        entry.configure(cursor="xterm")

    def add_choice(self, parent, label, variable):
        ttk.Label(parent, text=label, style="PanelTitle.TLabel").pack(anchor="w", pady=(7, 4))
        choice = ttk.Combobox(parent, textvariable=variable, values=["1", "2", "3", "4", "5"], state="readonly")
        choice.pack(fill="x")

    def build_dashboard(self, parent):
        right = ttk.Frame(parent)
        right.grid(row=0, column=1, sticky="nsew")
        right.columnconfigure(0, weight=1)
        right.rowconfigure(0, weight=3)
        right.rowconfigure(1, weight=2)

        table_panel = ttk.Frame(right, style="Panel.TFrame", padding=14)
        table_panel.grid(row=0, column=0, sticky="nsew", pady=(0, 12))
        ttk.Label(table_panel, text="YOUR REVISION TOPICS", style="PanelTitle.TLabel").pack(anchor="w", pady=(0, 10))

        columns = ("subject", "topic", "days", "score", "priority", "status")
        self.tree = ttk.Treeview(table_panel, columns=columns, show="headings", selectmode="browse", height=6)
        headings = {"subject": "SUBJECT", "topic": "TOPIC", "days": "DAYS LEFT", "score": "SCORE", "priority": "PRIORITY", "status": "STATUS"}
        widths = {"subject": 105, "topic": 190, "days": 75, "score": 65, "priority": 85, "status": 90}
        for col in columns:
            self.tree.heading(col, text=headings[col])
            self.tree.column(col, width=widths[col], anchor="w" if col in ("subject", "topic") else "center")
        self.tree.tag_configure("HIGH", foreground="#fca5a5")
        self.tree.tag_configure("MEDIUM", foreground="#fcd34d")
        self.tree.tag_configure("LOW", foreground="#86efac")
        self.tree.pack(fill="both", expand=True)
        self.tree.bind("<<TreeviewSelect>>", self.on_select)

        actions = ttk.Frame(table_panel, style="Panel.TFrame")
        actions.pack(fill="x", pady=(10, 0))
        ttk.Button(actions, text="MARK COMPLETED", style="Secondary.TButton", command=self.mark_completed).pack(side="left", padx=(0, 7))
        ttk.Button(actions, text="REMOVE", style="Secondary.TButton", command=self.remove_topic).pack(side="left")
        ttk.Button(actions, text="GENERATE TODAY'S PLAN", style="Accent.TButton", command=self.generate_plan).pack(side="right")

        bottom = ttk.Frame(right)
        bottom.grid(row=1, column=0, sticky="nsew")
        bottom.columnconfigure(0, weight=1)
        bottom.columnconfigure(1, weight=1)
        bottom.rowconfigure(0, weight=1)

        recommendation = ttk.Frame(bottom, style="Panel.TFrame", padding=14)
        recommendation.grid(row=0, column=0, sticky="nsew", padx=(0, 7))
        ttk.Label(recommendation, text="AI RECOMMENDATION", style="PanelTitle.TLabel").pack(anchor="w", pady=(0, 8))
        self.recommendation_text = tk.Text(recommendation, height=7, wrap="word", bg="#111c2f", fg="#cbd5e1", insertbackground="#ffffff",
        relief="flat", font=("Arial", 9), padx=2, pady=2)
        self.recommendation_text.pack(fill="both", expand=True)
        self.recommendation_text.insert("1.0", "Select a topic to view its priority score, the reasons behind the result and a suggested next
        step.")
        self.recommendation_text.configure(state="disabled")

        plan = ttk.Frame(bottom, style="Panel.TFrame", padding=14)
        plan.grid(row=0, column=1, sticky="nsew", padx=(7, 0))
        ttk.Label(plan, text="TODAY'S STUDY PLAN", style="PanelTitle.TLabel").pack(anchor="w", pady=(0, 8))
        self.plan_text = tk.Text(plan, height=7, wrap="word", bg="#111c2f", fg="#cbd5e1", insertbackground="#ffffff", relief="flat",
        font=("Arial", 9), padx=2, pady=2)
        self.plan_text.pack(fill="both", expand=True)
        self.plan_text.insert("1.0", "Add or load topics, then choose Generate Today's Plan.")
        self.plan_text.configure(state="disabled")

```

# 4. SOURCE CODE - CONTINUED

StudyWise_AI_Project.py

```
    def load_topics(self):
        if not DATA_FILE.exists():
            return []
        try:
            data = json.loads(DATA_FILE.read_text(encoding="utf-8"))
            return data if isinstance(data, list) else []
        except (OSError, json.JSONDecodeError):
            messagebox.showwarning("Data file", "The saved data file could not be read. A fresh list will be started.")
            return []

    def save_topics(self):
        try:
            DATA_FILE.write_text(json.dumps(self.topics, indent=2), encoding="utf-8")
        except OSError as error:
            messagebox.showerror("Save error", f"Could not save your topics:\n{error}")

    def add_topic(self):
        subject = self.subject_var.get().strip()
        topic = self.topic_var.get().strip()
        if not subject or not topic:
            messagebox.showwarning("Missing information", "Please enter both a subject and a topic.")
            return
        try:
            days = int(self.days_var.get())
            difficulty = int(self.difficulty_var.get())
            confidence = int(self.confidence_var.get())
            importance = int(self.importance_var.get())
            if days < 0:
                raise ValueError("Days left must be zero or more.")
            result = analyse_topic(days, difficulty, confidence, importance)
        except ValueError as error:
            messagebox.showerror("Check your values", str(error))
            return

        new_id = max((int(item.get("id", 0)) for item in self.topics), default=0) + 1
        self.topics.append({
            "id": new_id,
            "subject": subject,
            "topic": topic,
            "days": days,
            "difficulty": difficulty,
            "confidence": confidence,
            "importance": importance,
            "score": result["score"],
            "priority": result["priority"],
            "advice": result["advice"],
            "explanation": result["explanation"],
            "completed": False,
        })
        self.save_topics()
        self.refresh_topics()
        self.subject_var.set("")
        self.topic_var.set("")
        self.show_recommendation(self.topics[-1])

    def refresh_topics(self):
        for row in self.tree.get_children():
            self.tree.delete(row)
        ordered = sorted(self.topics, key=lambda item: (item.get("completed", False), -item.get("score", 0)))
        for item in ordered:
            status = "COMPLETED" if item.get("completed") else "PENDING"
            self.tree.insert("", "end", iid=str(item["id"]), values=(
                item["subject"], item["topic"], item["days"], f"{item['score']:.2f}", item["priority"], status
            ), tags=(item["priority"],))
        total = len(self.topics)
        high = sum(1 for item in self.topics if item["priority"] == "HIGH" and not item.get("completed"))
        done = sum(1 for item in self.topics if item.get("completed"))
        self.stat_total.configure(text=str(total))
        self.stat_high.configure(text=str(high))
        self.stat_done.configure(text=str(done))
        self.stat_pending.configure(text=str(total - done))

    def get_selected_topic(self):
        selection = self.tree.selection()
        if not selection:
            return None
        selected_id = int(selection[0])
        return next((item for item in self.topics if int(item["id"]) == selected_id), None)

    def on_select(self, _event=None):
        item = self.get_selected_topic()
        if item:
            self.show_recommendation(item)

    def show_recommendation(self, item):
        text = (
            f"{item['priority']} PRIORITY  |  Score: {item['score']:.2f}/5\n"
            f"WHY: {item['explanation']}.\n"
            f"NEXT: {item['advice']}"
        )
        self.set_text(self.recommendation_text, text)
```

# 4. SOURCE CODE - CONTINUED

StudyWise_AI_Project.py

```
0361
0362      def mark_completed(self):
0363          item = self.get_selected_topic()
0364          if not item:
0365              messagebox.showinfo("Select a topic", "Choose a topic from the table first.")
0366              return
0367          item["completed"] = not item.get("completed", False)
0368          self.save_topics()
0369          self.refresh_topics()
0370          self.show_recommendation(item)
0371
0372      def remove_topic(self):
0373          item = self.get_selected_topic()
0374          if not item:
0375              messagebox.showinfo("Select a topic", "Choose a topic from the table first.")
0376              return
0377          self.topics = [entry for entry in self.topics if entry["id"] != item["id"]]
0378          self.save_topics()
0379          self.refresh_topics()
0380          self.set_text(self.recommendation_text, "Select a topic to view its recommendation.")
0381
0382      def generate_plan(self):
0383          try:
0384              available = int(self.minutes_var.get())
0385              if not 15 <= available <= 600:
0386                  raise ValueError
0387          except ValueError:
0388              messagebox.showerror("Study time", "Enter a daily study time between 15 and 600 minutes.")
0389              return
0390          pending = sorted(
0391              (item for item in self.topics if not item.get("completed")),
0392              key=lambda item: item["score"],
0393              reverse=True,
0394          )
0395          lines = [f"TIME: {available} minutes"]
0396          used = 0
0397          for item in pending:
0398              duration = {"HIGH": 40, "MEDIUM": 25, "LOW": 15}[item["priority"]]
0399              if used + duration > available:
0400                  continue
0401              used += duration
0402              lines.append(f"{duration}m | {item['subject']}: {item['topic']} ({item['priority']})")
0403          if not pending:
0404              lines.append("All saved topics are marked completed.")
0405          elif used == 0:
0406              lines.append("No complete session fits. Increase the daily minutes.")
0407          else:
0408              lines.append(f"Planned: {used} / {available} minutes. Take short breaks between sessions.")
0409          self.set_text(self.plan_text, "\n".join(lines))
0410
0411      def set_text(self, widget, value):
0412          widget.configure(state="normal")
0413          widget.delete("1.0", "end")
0414          widget.insert("1.0", value)
0415          widget.configure(state="disabled")
0416
0417      def load_demo_data(self):
0418          samples = [
0419              ("Mathematics", "Quadratic Equations", 3, 5, 2, 5),
0420              ("Science", "Electricity", 7, 3, 3, 4),
0421              ("Social Science", "Nationalism in India", 18, 2, 4, 3),
0422          ]
0423          self.topics = []
0424          for index, (subject, topic, days, difficulty, confidence, importance) in enumerate(samples, start=1):
0425              result = analyse_topic(days, difficulty, confidence, importance)
0426              self.topics.append({
0427                  "id": index, "subject": subject, "topic": topic, "days": days,
0428                  "difficulty": difficulty, "confidence": confidence,
0429                  "importance": importance, "score": result["score"],
0430                  "priority": result["priority"], "advice": result["advice"],
0431                  "explanation": result["explanation"], "completed": False,
0432              })
0433          self.save_topics()
0434
0435      def load_demo_action(self):
0436          self.load_demo_data()
0437          self.refresh_topics()
0438          if self.topics:
0439              first = max(self.topics, key=lambda item: item["score"])
0440              self.tree.selection_set(str(first["id"]))
0441              self.show_recommendation(first)
0442
0443
0444  def main():
0445      root = tk.Tk()
0446      StudyWiseApp(root)
0447      root.mainloop()
0448
0449
0450  if __name__ == "__main__":
0451      main()
```

# 5. OUTPUT

The following screenshot shows the StudyWise AI desktop dashboard with three sample topics. The table displays each topic's score, priority and completion status. The four summary cards show the number of topics, high-priority topics, completed topics and topics still to revise.

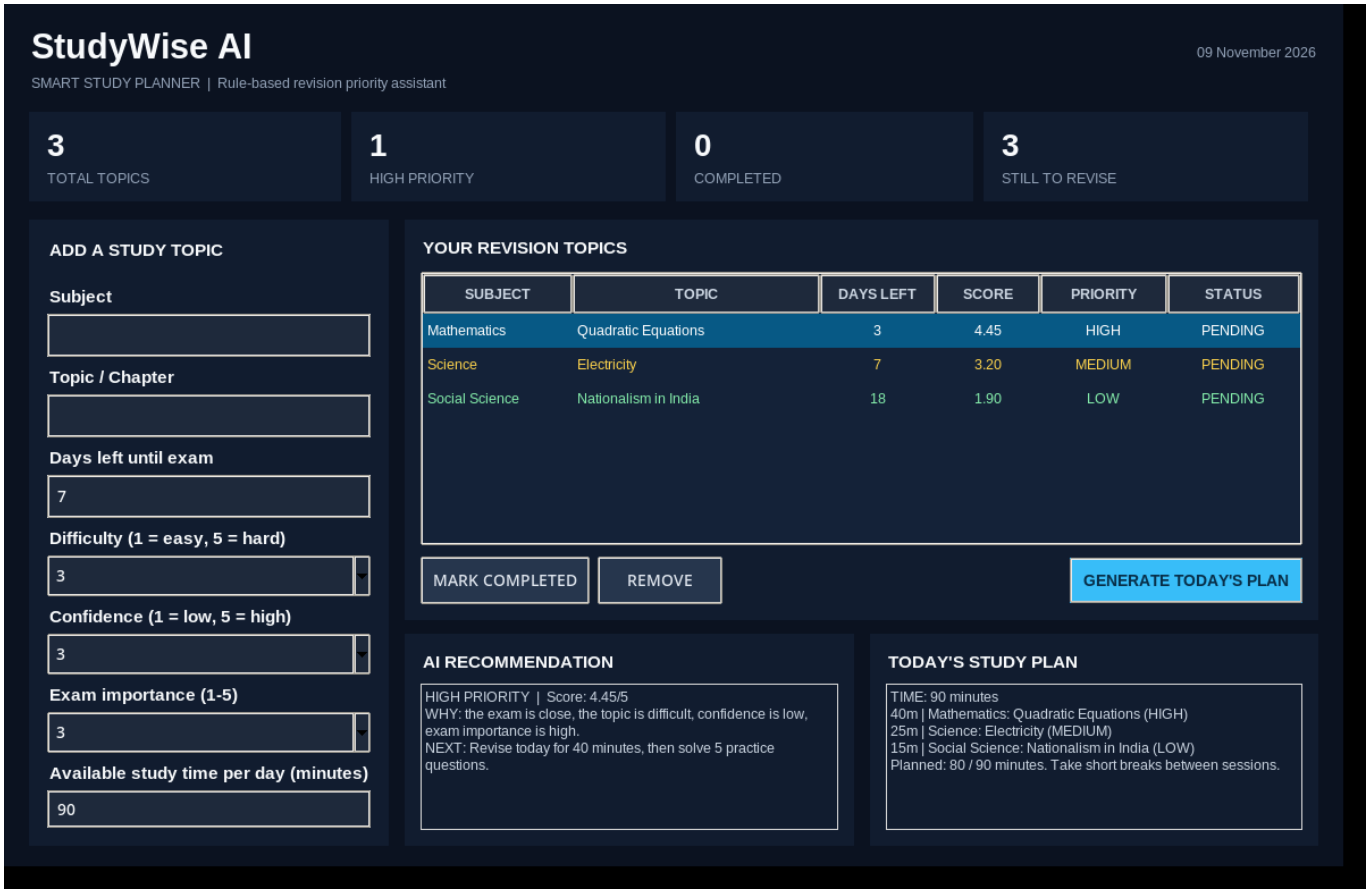



*Figure 1. StudyWise AI dashboard showing the sample topic list and priority scores.*

**Sample results shown.** Mathematics - Quadratic Equations: 4.45 (HIGH); Science - Electricity: 3.20 (MEDIUM); Social Science - Nationalism in India: 1.90 (LOW).

# 5. OUTPUT - RECOMMENDATION AND PLAN

Figure 2 shows the selected topic's explanation and a generated daily study plan. The plan assigns a suggested session length according to priority, then checks that the total does not exceed the student's available time.

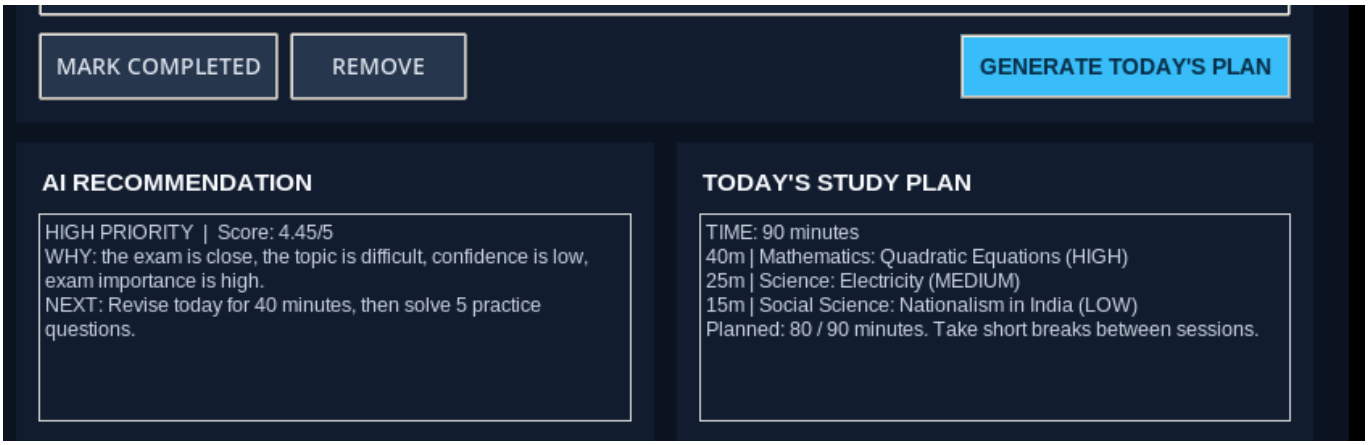



*Figure 2. AI recommendation and the generated 90-minute study plan.*

### TESTING AND OUTPUT INTERPRETATION

| Days, Difficulty, Confidence, Importance | Score / 5 | Result |
|---|---|---|
| 3, 5, 2, 5 | 4.45 | HIGH |
| 7, 3, 3, 4 | 3.20 | MEDIUM |
| 18, 2, 4, 3 | 1.90 | LOW |

**Interpretation.** A smaller number of days, higher difficulty, lower confidence and higher exam importance increase the priority score. The output is a revision-planning suggestion, not a prediction of examination marks.

**Input validation.** Negative days are rejected, and difficulty, confidence and importance must stay between 1 and 5. Daily study time must be between 15 and 600 minutes.

# 6. CONCLUSION

StudyWise AI demonstrates how a practical student problem can be addressed with a simple Python application. It accepts study information, applies a transparent rule-based decision system, ranks topics by priority and creates a daily plan that respects the entered time limit.

**What the project demonstrates.**

- A graphical desktop interface created with Tkinter, with Python as the core language.
- Input validation and local JSON storage using Python standard-library modules.
- A weighted decision score with explainable HIGH, MEDIUM and LOW categories.
- A daily plan that fits the available study time.

**Limitations.** StudyWise AI is a rule-based educational prototype. It does not train a machine-learning model, does not know a student's actual learning ability, and cannot guarantee marks or academic results. Its recommendations depend on the accuracy of the values entered by the user.

**Future scope.** Future versions could include weekly calendar views, subject-wise progress charts, export options, reminders and user-adjustable scoring weights. Any future data-driven model should be tested carefully and should protect students' personal information.

### FINAL NOTE

The goal is to make revision more organised, understandable and manageable.

# 7. REFERENCES


- **1.** Central Board of Secondary Education (CBSE), Artificial Intelligence (Subject Code 417), Class X curriculum resource. Official CBSE curriculum PDF.
- **2.** Python Software Foundation, *tkinter - Python interface to Tcl/Tk*. Python documentation.
- **3.** Python Software Foundation, *json - JSON encoder and decoder*. Python documentation.
- **4.** Python Software Foundation, *datetime - Basic date and time types*. Python documentation.
- **5.** Python Software Foundation, *pathlib - Object-oriented filesystem paths*. Python documentation.


## HOW TO RUN THE PROJECT

- 1. Install Python 3 and make sure Tkinter is available.
- 2. Save StudyWise_AI_Project.py in a folder.
- 3. Open a terminal in that folder and run: python3 StudyWise_AI_Project.py
- 4. Use LOAD SAMPLE TOPICS to view the demonstration, or enter your own topics.
- 5. Your topics are saved in studywise_data.json in the same folder.

### IMPORTANT TECHNICAL NOTE

Python is the core language in this desktop version; Tkinter provides its GUI.

HTML/CSS/JavaScript may be added for a future web version; they are optional.